TP5 — Convolution rapide par FFT

TP

Guilhem PAGES **Version 1**
guilhem.pages@univ-lemans.fr **Durée : 1 h 40**

Problématique

Au TP4, la convolution directe coûte M multiplications par échantillon : au-delà de quelques milliers de coefficients, le temps réel est perdu. Or filtrer, c'est aussi *multiplier les spectres*. Comment utiliser la FFT pour convoluer un flux continu, bloc par bloc, sans aucune erreur, et à partir de quelle longueur de filtre est-ce rentable ?

Objectifs:

- Manipuler la FFT et les nombres complexes (`std::complex<float>`) sur la Teensy ;
- Distinguer convolution **circulaire** (TFD) et convolution **linéaire** (filtrage), et choisir la taille de FFT $N \geq L + M - 1$ (**zéro-padding**) ;
- Implémenter l'**overlap-save** (recouvrement des entrées) puis l'**overlap-add** (recouvrement des sorties), et les comparer ;
- Valider le résultat à l'oscilloscope : même réponse impulsionnelle qu'au TP4 ;
- Comparer les coûts mesurés et théoriques, et trouver la longueur de rentabilité.

Prérequis:

- TP4 terminé (convolution directe, buffer circulaire, lecture d'une réponse impulsionnelle) ;
- TFD, propriété « convolution ↔ produit », FFT (TNS hors-ligne).

® **Remarque** : Projets Arduino

Deux projets, chacun en version `etudiant` et `corrige` :

- `jour2/TP5a_fft_overlap_save` : convolution par FFT, overlap-save (TP) ;
- `jour2/TP5b_fft_overlap_add` : convolution par FFT, overlap-add (TP).

Les FFT sont celles de la bibliothèque **CMSIS-DSP** d'ARM, livrée avec Teensyduino ; le fichier fourni `fft_cmsis.h` rassemble ce qu'il faut pour s'en servir.

® **Remarque** : Montage et réglages

Oscilloscope : voie 1 sur `LINE OUT` gauche (signal traité), voie 2 sur `LINE OUT` droite (source non traitée), déclenchement sur la voie 2. Tous les réglages se font dans `setup()`, dans le fichier `.ino`, puis on téléverse à nouveau (une vingtaine de secondes) :

- **source** : gain 1 sur une seule voie du mélangeur `source` : *impulsions* (une toutes les 100 ms : la voie 1 montre directement $h[n]$), *sinus* de test, ou *musique* du PC par USB (*Outils* → *USB Type* → *« Serial + MIDI + Audio »*) ;
- **réponse impulsionnelle** : comme au TP4, garder une seule ligne `int M = noyau...(h, ...);` ;
- **charge processeur** : affichée chaque seconde dans le moniteur série, en pourcentage de la durée d'un bloc (2,9 ms).

1 Cours éclair : de la FFT à la convolution rapide

TFD et FFT. La TFD de N points d'un vecteur x est $X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}$; la case k correspond à la fréquence $k\,F_s/N$. La FFT la calcule en $\frac{N}{2}\log_2 N$ multiplications complexes au lieu de N^2.

CMSIS-DSP. ARM fournit avec ses processeurs Cortex-M une bibliothèque de traitement du signal optimisée : FFT, filtres FIR et IIR, opérations sur des vecteurs… On ne réécrit pas une FFT : on utilise `arm_cfft_f32`, déclarée dans `arm_math.h`.

Les nombres complexes en C++. Le fichier fourni `fft_cmsis.h` définit le type `complex` (c'est `std::complex<float>`) : un nombre complexe dont les parties réelle et imaginaire sont des `float`.

`complex(a, b)`	construit $a + jb$; un échantillon réel x s'écrit `complex(x, 0.0f)`
`X[k] * H[k]`	produit de deux complexes
`z.real()`, `z.imag()`	parties réelle et imaginaire
`std::abs(z)`, `std::arg(z)`	module et phase

Les deux transformées, pour N puissance de 2 (128 à 4096) :

```
fft_ = instanceFFT(N);                                    // tables de la FFT de N points
arm_cfft_f32(fft_, (float *)x, FFT_DIRECTE, ORDRE_NATUREL);  // x devient sa TFD
arm_cfft_f32(fft_, (float *)x, FFT_INVERSE, ORDRE_NATUREL);  // TFD inverse (avec 1/N)
```

Le calcul se fait *sur place* : le résultat remplace les données, il n'y a pas de tableau de sortie. CMSIS attend un tableau de `float` (ré_0, im_0, ré_1, im_1, …) : c'est exactement la façon dont un tableau de `complex` est rangé en mémoire, d'où la conversion `(float *)x`.

Convolution circulaire. Si $Y[k] = H[k]\,X[k]$ pour $k = 0..N-1$, alors

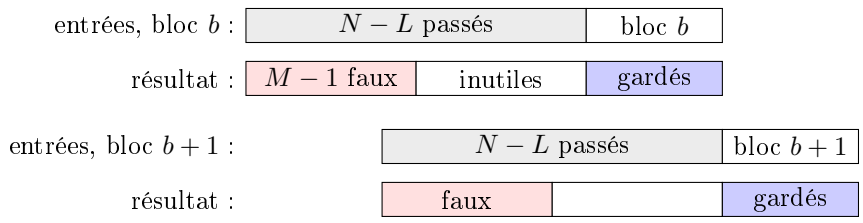
$$y[n] = \sum_{k=0}^{N-1} h[k]\; x\big[(n-k) \bmod N\big] :$$

la TFD voit ses N points comme *une période* d'un signal périodique. Ce qui « déborde » à droite revient à gauche : c'est le **repliement circulaire**.

Overlap-save (OLS) : on recouvre les entrées. Comme l'historique du TP4a, les entrées sont rangées dans un buffer circulaire. À chaque bloc, on transforme les N *derniers* échantillons reçus : les $N-L$ précédents, suivis du bloc courant. Le noyau h est complété par des zéros jusqu'à N points. Après produit des spectres et FFT inverse, la valeur n du résultat a besoin des entrées $n, n-1, \ldots, n-M+1$:

- pour $n \geq M-1$, elles sont toutes dans les N points transformés : la valeur est exacte ;
- pour $n < M-1$, la convolution circulaire va chercher les entrées manquantes à la *fin* du tampon (repliement) : ces $M-1$ premières valeurs sont fausses.

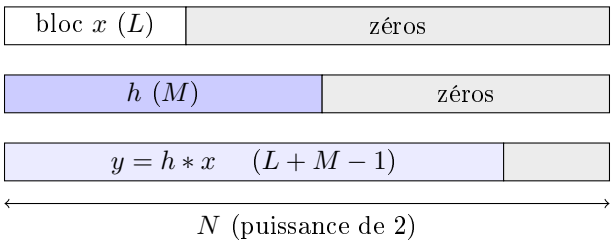
On ne garde donc que les L *dernières* valeurs, exactes si $N - L \geq M - 1$, soit $N \geq L + M - 1$. Au bloc suivant, la fenêtre de N points avance de L.



Zéro-padding (pour l'overlap-add). La convolution linéaire d'un bloc de L points par h de M points compte $L + M - 1$ points. Si on complète bloc et noyau par des zéros jusqu'à

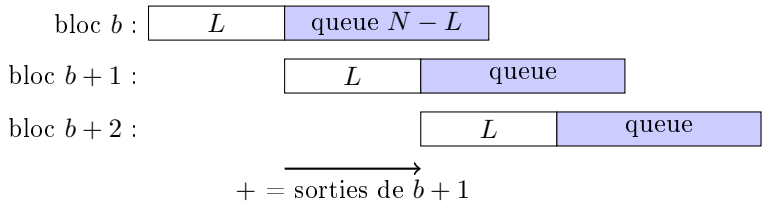
$$N \;\geq\; L + M - 1,$$

rien ne déborde : la convolution circulaire *est* la convolution linéaire.



Overlap-add (OLA) : on recouvre les sorties. Chaque bloc filtré fait $L + M - 1$ points, mais seuls L sortent maintenant. Les $N - L$ derniers forment une **queue** qu'on ajoute aux sorties des blocs suivants :

$$\text{sortie}[n] = y_{\text{courant}}[n] + \text{queue}[n], \qquad n = 0..L-1.$$



Dans le code, comme au TP4, les sorties en cours d'accumulation sont rangées dans un **buffer circulaire** `acc_` : on y ajoute les N valeurs du bloc filtré à partir de l'index r, les L premières sortent, leurs cases sont remises à zéro, et r avance de L. La queue reste en place : rien n'est jamais déplacé.

Coût. Par bloc : une FFT, N produits complexes, une FFT inverse (la FFT de h est calculée une fois pour toutes, dans `setup()`). Une multiplication complexe vaut 4 multiplications réelles, soit par échantillon :

$$C_{\text{FFT}} \approx \frac{4}{L}\left(2 \cdot \tfrac{N}{2}\log_2 N + N\right) \qquad \text{contre} \qquad C_{\text{directe}} = M.$$

2 Filtrer dans le domaine fréquentiel (projet TP5a)

Activité 1

Q1. Dans `conv_fft_ols.cpp`, compléter `setKernel()` (TODO Q1) : choisir N, puis calculer $H = \text{FFT}(h)$ sur N points (zéro-padding de h). Donner N pour $M = 16, 256, 1024$ et 3968. Pourquoi ce calcul est-il fait dans `setup()`, avant `AudioMemory()`, et pas dans `update()` ?

Q2. Écrire le produit des spectres (TODO Q2). *Sur papier :* $N = 4$, $x = (1, 2, 3, 4)$, $h = (1, 1)$; calculer la convolution circulaire et la convolution linéaire. Quel échantillon est faux ? Combien de valeurs fausses y a-t-il pour un noyau de M coefficients ?

```
void AudioFilterConvOLS::setKernel(const float *h, int M) {
  // TODO (Q1) : taille de FFT N = plus petite puissance de 2 >= L + M - 1
  //             (fonction puissance2() de fft_cmsis.h)
  const int N = 128;                    // provisoire

  // TODO (Q1) : H_[n] = h[n] pour n < M, 0 au-delà (zéro-padding), puis H = FFT(h) :
  //               fft_ = instanceFFT(N);
  //               arm_cfft_f32(fft_, (float *)H_, FFT_DIRECTE, ORDRE_NATUREL);
  for (int n = 0; n < N; n++) H_[n] = complex(1.0f, 0.0f);    // provisoire : H = 1

  N_ = N;
  M_ = M;
}
```

```
// passage en fréquence : FFT de CMSIS-DSP, sur place
arm_cfft_f32(fft_, (float *)X_, FFT_DIRECTE, ORDRE_NATUREL);

// TODO (Q2) : filtrage = produit des spectres X_[k] · H_[k], case par case

// retour en temps : FFT inverse (division par N comprise)
arm_cfft_f32(fft_, (float *)X_, FFT_INVERSE, ORDRE_NATUREL);
```

3 Overlap-save

Activité 2

Q3. Le code provisoire transforme le bloc seul, complété par des zéros, sans aucune mémoire. Source « impulsions » : observer à l'oscilloscope la réponse au noyau `noyauRetard(h, 2000)`, puis au noyau `noyauSalle(h, 3968)`. Que manque-t-il ? Compléter les trois TODO (Q3) : écriture du bloc dans le buffer circulaire `entree_`, copie des N dernières entrées dans `X_`, sortie des L dernières valeurs. Valider : pour plusieurs noyaux, la voie 1 doit montrer la même réponse impulsionnelle qu'au TP4a.

```
// TODO (Q3) 1) écrire les L échantillons du bloc (normalisés) dans entree_[w_],
//              en faisant avancer w_ avec bouclage (& FFT_MASK)

// TODO (Q3) 2) copier les N dernières entrées dans X_ :
//              X_[i] = complex(entree_[(w_ - N + i) & FFT_MASK], 0.0f)
// provisoire : le bloc seul, complété par des zéros (aucune mémoire)
for (int n = 0; n < CONV_L; n++) X_[n] = complex(block->data[n] / 32768.0f, 0.0f);
for (int n = CONV_L; n < N; n++) X_[n] = complex(0.0f, 0.0f);
```

```
// TODO (Q3) 3) garder les L DERNIÈRES valeurs : sortie n = X_[N - L + n].real()
for (int n = 0; n < CONV_L; n++) block->data[n] = versInt16(X_[n].real());   // provisoire
```

4 Coût : quand la FFT est-elle rentable ?

Activité 3

Q4. Source « musique », noyau `noyauSalle(h, M)`. Pour chaque M du tableau, relever la charge du TP5a (FFT) et celle du TP4a (directe). Calculer C_{FFT} et C_{directe} (multiplications réelles par échantillon). Où se situe la longueur de rentabilité théorique ? mesurée ? Pourquoi le gain reste-t-il modeste ici ? Combien coûterait un écho de 1 s par FFT ?

M	16	128	256	1024	2048	3968
N						
CPU FFT (%)						
CPU directe (%)						

5 Overlap-add (projet TP5b)

Activité 4

Q5. Dans `conv_fft_ola.cpp`, `setKernel()` et le produit des spectres sont fournis. Compléter l'overlap-add dans le buffer circulaire `acc_` (TODO Q5) et valider comme en Q3. Comparer avec l'overlap-save : quelle méthode utilise le moins de mémoire ? d'opérations ?

```
// TODO (Q5) 5) overlap-add dans le buffer circulaire acc_ :
//   - ajouter les N valeurs X_[i].real() dans acc_[(r_ + i) & FFT_MASK]
//   - pour n = 0 .. L-1 : envoyer acc_[(r_ + n) & FFT_MASK] (versInt16),
//     puis remettre cette case à 0
//   - faire avancer r_ de L (avec bouclage)
for (int n = 0; n < CONV_L; n++) block->data[n] = versInt16(X_[n].real());   // provisoire : sans la queue
```

6 Livrables

1. Convolution FFT en overlap-save puis en overlap-add, validées à l'oscilloscope contre le TP4a ;
2. Calcul papier de la convolution circulaire (valeurs repliées) ;
3. Tableau de charge FFT / directe et longueur de rentabilité, comparées à la théorie.

® **Remarque** : Pour aller plus loin

Exploiter la symétrie hermitienne : le signal étant réel, une FFT complexe de $N/2$ points suffit (ou deux blocs réels dans une même FFT complexe). Comparer avec `arm_rfft_fast_f32` de la bibliothèque CMSIS-DSP. Implémenter une convolution partitionnée uniforme (noyau découpé en P morceaux de 128 points).